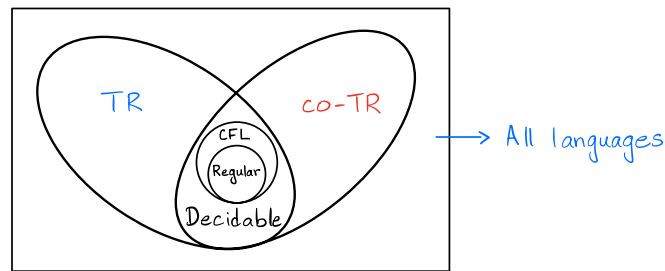


co-Turing Recognizable (TR) / co-Recursively Enumerable

A language L is co-TR if there exists a TM M such that for all strings $w \notin L$, TM M halts & rejects. (May loop or accept other strings)

Decidable / Recursive

A language is decidable if it is both TR & co-TR.



UNIVERSAL TURING MACHINE (UTM)

UTM M_u takes as input encoding of TM M (denoted as $\langle M \rangle$ OR $e(M)$) & input string w encoded as $\langle w \rangle$ OR $e(w)$. M_u simulates M on w , i.e.,

$$M_u(\langle M, w \rangle) = \begin{cases} \text{accept} & \text{if } w \in L(M) \\ \text{reject} & \text{if } w \notin L(M) \\ \text{loop} & M \text{ loops on } w \end{cases}$$

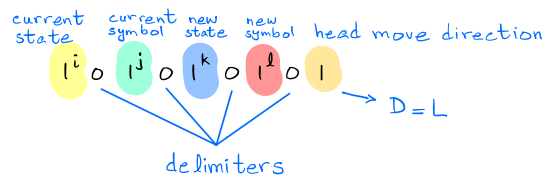
Let us fix the encoding of the input
Fix $\Sigma = \{0, 1\}$ for encoding of M

- $Q = \{q_1, q_2, \dots, q_n\}$
 - q_1 — start state
 - q_2 — accept state
 - q_3 — reject state
- $\Gamma = \{\gamma_1, \gamma_2, \dots, \gamma_t\}$
 - γ_1 — Blank symbol (B)

Encoding of transition function

$$\delta(q_i, \gamma_j) = (q_k, \gamma_l, D) \text{ , } D \in \{L, R, S\}$$

$L = |$
 $R = ||$
 $S = |||$



The sequence of transitions is delimited by 00. This entire sequence is the encoding $e(M)$.

We assume M has already been transformed into our standard 7-tuple, if it was a variant TM.

String $w = x_{i_1} x_{i_2} \dots x_{i_x}$ where $2 \leq i_j \leq t$ for $1 \leq j \leq x$ → starts from 2 since B is not in w

So, $e(w) = i_1^1 0 i_2^2 0 \dots i_x^x$

Input to M_u is $e(M) 000 e(w)$

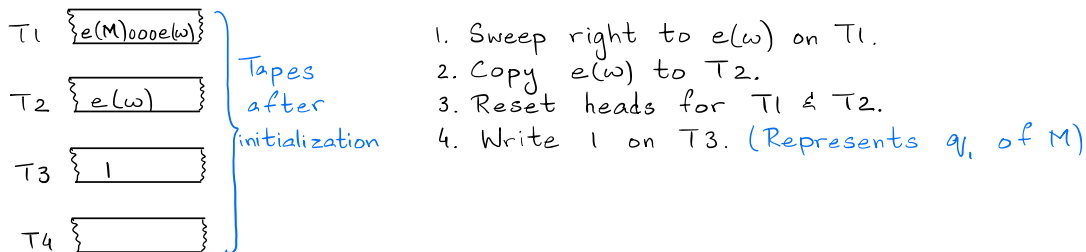
Tapes of M_u

Define M_u to have 4-tapes. Just for presentation clarity.
Tapes 2, 3 & 4 are initially empty.

1. Tape 1 (T_1) has input $e(M) 000 e(w)$.
2. Tape 2 (T_2) will serve as the tape of M , i.e., contents will reflect how M would update its own tape on input w .
3. Tape 3 (T_3) stores current state of M , i.e., i^j if current state is q_i .
4. Tape 4 (T_4) Workspace for M_u .

Algorithm for Simulation

Initialization



Simulation

1. Copy current state from T_3 to T_4 . Just 1 on first iteration for q_1 .
2. Write 0 after i^j on T_4 .
3. Copy current sequence of 1's pointed to by head of T_2 to T_4 .
In first iteration its i^1 for x_{i_1}
4. Write 0 at end of sequence of T_4 .
T4 now contains $1 0 i^1 0$ on first iteration.
5. Sweep right on T_1 till we match a prefix of transition function entry with T_4 contents.
This is $\delta(q_i, x_{i_j})$. Need to look up this sequence in $e(M)$ on T_1 .
6. Update T_3 content to i^j . If $j=2$, accept. If $j=3$ reject.
7. Update current sequence of i^j on T_2 to i^k . Shift string as needed.
8. If $j=1$, move T_2 to sequence of 1's on left
If $j=2$, move T_2 to sequence of 1's on right
9. Clear T_4 & goto step 1.

THE HALTING PROBLEM (HP)

Can we use M_u to decide every language? No!

HP: Given $\langle M, w \rangle$ as input decide whether M halts on w .

If we could decide HP, can use its decider P to trivialize all of mathematics. How?

e.g.,

Goldbach's Conjecture

All even numbers ≥ 4 are sum of two primes.

We can make the following checker TM:

M :

```
while  $i \geq 2$ 
  if  $2i$  is not a sum of 2 primes then
    HALT
```

Since we assume P decides HP, can run $P(\langle M, \lambda \rangle)$

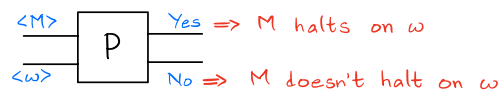
$$P(\langle M, \lambda \rangle) = \begin{cases} \text{Yes} \Rightarrow M \text{ halts} \Rightarrow \text{Goldbach's conjecture is False} \\ \text{No} \Rightarrow M \text{ doesn't halt} \Rightarrow \text{Goldbach's conjecture is True} \end{cases}$$

Can similarly build a way to prove/disprove any conjecture!

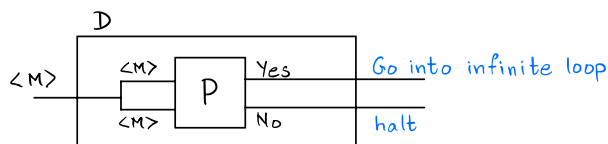
Proof that HP is not decidable

Assume for sake of contradiction HP is decidable.

Then, there exists a program P that decides HP, i.e.,



We build the following program D that uses P as a subroutine.



Now, consider running D on input $\langle D \rangle$, i.e., $D(\langle D \rangle)$.

Complete your argument here for the contradiction.

Equivalent Pseudocode

```
 $D(\langle M \rangle)$ :  
   $i = 1$   
  if  $P(\langle M \rangle, \langle M \rangle)$  then  
    while  $(i == 1)$  { }  
  else  
    return
```

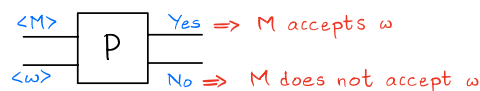
Membership Problem (MP)

$A_{TM} = \{ \langle M, w \rangle \mid M \text{ is a valid TM encoding \& } M \text{ accepts } w \}$

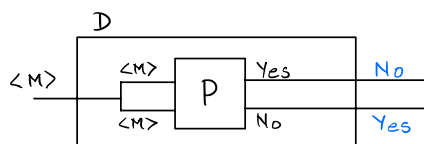
Note $A_{DFA} = \{ \langle M, w \rangle \mid M \text{ is a valid DFA encoding \& } M \text{ accepts } w \}$ is decidable.

Assume for sake of contradiction MP is decidable.

Then, there exists a program P that decides MP, i.e.,



We build the following program D that uses P as a subroutine.



Now, consider running D on input <D>, i.e., $D(\langle D \rangle)$.

Complete your argument here for the contradiction.

[Link to Cantor's diagonalization.](#)

Ordering of all countably infinite TMs.

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	...	$\langle M_j \rangle$	...
M_1					
M_2					
M_3					
$\vdots$					
M_i					
$\vdots$					

(i,j)
Y/N

If MP was decidable, we can fill in the infinite table above where entry (i,j) is Y if M_i accepts $\langle M_j \rangle$ as input & N otherwise.

Consider what the entry is for $M_i = D$ & $\langle M_j \rangle = \langle D \rangle$.